pontIA

APUNTES DE CLASE

Modelos de Generación de Imagen

6 de octubre de 2026

01 Entrenamiento clásico de un modelo

- **Inicialización de parámetros**
  Para entrenar un modelo de machine learning se parte de un modelo sin entrenar. Los parámetros (o pesos) de la red neuronal se inicializan con valores aleatorios. Hay variantes como seguir una distribución normal o poner algunos a cero, pero la base es aleatoria.
- **Proceso de entrenamiento**
  Los parámetros se van actualizando iterativamente hasta alcanzar un estado óptimo en el que el modelo resuelve la tarea de la mejor manera posible.
- **Parámetros congelados e inferencia**
  Tras el entrenamiento los parámetros quedan fijos y constantes. El modelo se guarda y se reutiliza para hacer inferencia: se le da un input, se pasa por los parámetros fijos y devuelve un output. Si no funciona bien, se vuelve al punto de partida y se reentrena. Esto aplica a cualquier modelo de machine learning, no solo a modelos de difusión.

> IDEA CLAVE
> **En el entrenamiento clásico los parámetros se inicializan con valores aleatorios, se ajustan durante el entrenamiento y quedan congelados para hacer inferencia.**

02 Fine tuning (ajuste fino)

- **Diferencia con entrenamiento clásico**
  En fine tuning se parte de un modelo ya pre-entrenado por un tercero. La inicialización de los pesos no es aleatoria: los parámetros ya tienen valores concretos que codifican conocimiento previo.
- **Objetivo del ajuste fino**
  Se ajustan ligeramente los parámetros para variar el comportamiento del modelo y especializarlo en una nueva tarea, sin perder el conocimiento adquirido en el pre-entrenamiento. Se puede entrenar con un dataset pequeño.
- **Ejemplo típico**
  Un proveedor como OpenAI o Google entrena un modelo de texto grande que aprende sintaxis, significado y relaciones entre palabras. Luego se fine tunea ese modelo para especializarlo en un ámbito concreto (medicina, abogacía, etc.). El resultado es un modelo con pesos congelados que se comporta de forma similar al original pero resuelve mejor los casos del dominio especializado.

PontIA

- **RAG vs fine tuning**
  Ambas técnicas permiten que un modelo base trabaje sobre dominios que no estaban en su entrenamiento (p. ej., documentos internos de una empresa). RAG busca documentación relevante y la pega como contexto en la pregunta de forma dinámica, en cada llamada. Fine tuning entrena el modelo con esos documentos y actualiza sus parámetros, de modo que el conocimiento queda integrado en el modelo. En RAG nunca se actualizan los parámetros del modelo.

- **Aplicación al curso**
  En los ejercicios de hoy se aplica fine tuning a modelos de generación de imagen para que produzcan siempre un estilo concreto (cubismo, hiperrealismo, abstracto, etc.). En modelos grandes basta con indicar el estilo en el prompt, pero en modelos pequeños no funciona y es necesario fine tunear.

> IDEA CLAVE
> **En fine tuning se parte de un modelo pre-entrenado (pesos con valor, no aleatorios) y se ajustan ligeramente para especializarlo en una nueva tarea, aprovechando el conocimiento ya adquirido.**

> IDEA CLAVE
> **RAG y fine tuning sirven para ampliar el conocimiento de un modelo a dominios nuevos, pero RAG inyecta contexto dinámicamente en el prompt sin tocar los parámetros, mientras que fine tuning actualiza los pesos del modelo.**

PontIA

03 Transfer learning

- **Diferencia con fine tuning**
  En transfer learning se parte también de un modelo preentrenado, pero se congelan la mayoría de las capas y solo se dejan entrenables la última o últimas capas. El resultado es un modelo con la mayoría de pesos estáticos y solo algunos actualizados.

- **Ventaja e inconveniente**
  El entrenamiento es más rápido (p. ej., actualizar 500.000 parámetros de la última capa en vez de 3.000 millones). Sin embargo, es menos flexible: al entrenar solo una capa, el comportamiento del modelo cambia muy ligeramente respecto al modelo base.

> IDEA CLAVE
> **En transfer learning se congelan la mayoría de capas y solo se reentrenan las últimas; es más rápido que fine tuning pero menos flexible.**

04 Repaso del modelo de difusión

- **Entrenamiento**
  La UNet recibe una imagen con ruido y el timestep en que se añadió ese ruido. Predice cuánto ruido contiene la imagen. Se compara con el ruido real, se obtiene el error y se repite. No hay encoder/decoder ni condicionamiento por texto en esta configuración.

- **Inferencia**
  Se parte de ruido completamente aleatorio con t = T (p. ej., T = 50). El modelo predice el ruido, se resta de la imagen y se repite paso a paso hasta obtener la imagen final. Si hay condicionamiento por texto, se pasa el texto como input adicional.

05 Ejercicio 1: fine tuning de un modelo de difusión sin texto

- **Librerías necesarias**
  `diffusers` (librería de Hugging Face para modelos de difusión), `datasets` (librería de Hugging Face para datasets) y `accelerate` (optimización del entrenamiento). Se usa GPU para acelerar.
- **Modelo base**
  Se usa un modelo de Google entrenado solo con caras de personas, sin condicionamiento por texto. Dado un ruido aleatorio, la UNet quita ruido en la dirección de caras humanas y genera una cara aleatoria distinta cada vez. No es una cara del entrenamiento, sino una imagen nueva generada por el modelo.
- **Pipeline DDPM**
  Se carga con `DDPMPipeline.from_pretrained(model_id)` en lugar de `StableDiffusionPipeline`. Esta tubería solo contiene el scheduler y la UNet, sin text encoder ni variational autoencoder. Se invoca sin parámetros (no hay prompt). Por defecto realiza 1.000 pasos de difusión hacia atrás (frente a los 50-100 del ejercicio anterior).
- **Objetivo y limitación**
  Se fine tunea este modelo con un dataset de Pokémon para intentar introducir rasgos de Pokémon en las caras generadas. El resultado no va a ser bueno: el dataset es muy pequeño y el cambio de dominio (de caras humanas a Pokémon) es demasiado grande para un entrenamiento a pequeña escala en Colab. En el segundo ejercicio se fine tunea Stable Diffusion con condicionamiento por texto y sí funciona.
- **Reducir timesteps en inferencia**
  Por defecto el modelo usa 1000 timesteps, lo que es muy lento. Se puede cargar solo el scheduler con su clase y actualizar el número de pasos con `set_timesteps(num_inference_steps=40)`. Con 40 pasos la calidad es casi igual y la generación es mucho más rápida.
- **Bucle de inferencia (generación)**
  Se itera sobre los timesteps del scheduler. En cada paso: 1) se prepara el input del modelo (ruido aleatorio X y timestep T), 2) se pasa por la UNet, que devuelve una predicción de ruido, 3) se resta el ruido predicho con `scheduler.step`, que devuelve `output.prev_sample`. En la primera iteración el input es ruido puro; a medida que se itera se van distinguiendo las formas hasta obtener la imagen final.
- **Dataset de Pokémon para fine tuning**
  Se usa un dataset de imágenes de Pokémon de Hugging Face. Como es generación sin condicionamiento por texto, no se necesita descripción ni pie de foto, solo las imágenes. Se carga con `load_dataset` indicando el split de train y se limita a 1000 ejemplos para no desbordar la memoria de Colab (el dataset completo tiene 7360 imágenes).
- **Preproceso de imágenes**
  Las imágenes deben adaptarse a las dimensiones de entrada del modelo preentrenado. Para saber cuáles son, se consulta la documentación del modelo (o su código fuente si no se indica). En este caso son 256×256. Se aplican tres transformaciones: resize a 256×256, conversión a tensor y normalización de píxeles para que el entrenamiento sea más estable.

PontIA

- **Qué es un tensor**
  Un vector es una lista de números (1 dimensión), una matriz tiene filas y columnas (2 dimensiones) y un tensor es cualquier estructura de 3 o más dimensiones. Las imágenes tienen 3 canales (RGB), así que al convertirlas a números se obtiene un tensor. Convertir la imagen a tensor es necesario para que el modelo pueda trabajar con ella.

- **Bucle de entrenamiento (fine tuning)**
  Se definen épocas, learning rate y optimizador Adam, igual que en el ejercicio anterior. El bucle externo recorre las épocas y el interno recorre las imágenes del dataset una a una. En cada iteración: 1) se genera ruido aleatorio con `torch.randn` del tamaño de las imágenes, 2) se elige un timestep T aleatorio con `torch.randint` entre 0 y el número de timesteps del scheduler, 3) se añade el ruido a la imagen limpia con `scheduler.add_noise(imagen, ruido, T)`, 4) se pasa la imagen con ruido y T por la UNet para obtener el ruido predicho, 5) se calcula el MSE entre el ruido predicho y el real, 6) se hace backward y se actualizan los parámetros.

- **Diferencia con el ejercicio anterior**
  La lógica del bucle es la misma que al entrenar un modelo desde cero. La diferencia es que aquí se parte de un pipeline preentrenado (en este caso de Google) y se ajusta con un dataset propio. La función `add_noise` ya viene incluida en el scheduler del pipeline, mientras que antes se construyó manualmente.

- **Resultados del fine tuning**
  Con la GPU de Colab el entrenamiento es insuficiente y los resultados son pobres: las imágenes generadas son una mezcla abstracta entre rasgos faciales del modelo original y formas de Pokémon. Para un entrenamiento que funcione de verdad haría falta más potencia de GPU. El objetivo del ejercicio es entender el concepto de fine tuning, no obtener resultados de calidad.

- **Paso a Stable Diffusion**
  El segundo notebook trabaja con Stable Diffusion, que introduce dos novedades respecto al modelo anterior: 1) es un modelo de difusión latente, que usa un variational autoencoder (VAE) y realiza el proceso de difusión en el espacio latente en lugar de en el de píxeles, lo que lo hace más rápido, y 2) permite condicionamiento por texto.

> IDEA CLAVE
> **El fine tuning consiste en coger un modelo preentrenado de un tercero y ajustarlo con un dataset propio; el bucle de entrenamiento es el mismo que el de un modelo de difusión desde cero.**

06 Ejercicio 2: fine tuning de Stable Diffusion con texto

- **Librerías necesarias**
  Se instalan `diffusers` (modelos de difusión de Hugging Face), `datasets` (datasets de Hugging Face), `accelerate` (eficiencia en entrenamiento) y `peft` (Parameter Efficient Fine Tuning), que permite hacer el fine tuning de forma más eficiente.

- **Modelo pre-entrenado**
  Se usa Stable Diffusion v1.4, desarrollado por el departamento de Computer Vision de una universidad de Múnich y subido a Hugging Face. Es mayor que los modelos anteriores, pero tiene un buen equilibrio entre tamaño y calidad de generación (más de medio millón de descargas mensuales).

PontIA

- **Carga del modelo**
  Se carga con `StableDiffusionPipeline.from_pretrained(model_id)` y se envía a GPU con `.to(device)` donde `device` es CUDA.

- **Prueba antes del fine tuning**
  Se genera una imagen con el prompt "an image of a sheep" usando 50 time steps y una semilla de reproducibilidad. El modelo genera una oveja realista. La semilla permite comparar el resultado antes y después del fine tuning con el mismo prompt.

- **Dataset con captions**
  Para fine tunear Stable Diffusion no vale un dataset solo con imágenes: al ser un modelo condicionado por texto, se necesitan parejas de imagen y descripción (caption). Durante el entrenamiento se le enseña: dada una imagen con ruido, el time step T y el texto que la describe, predecir cuánto ruido hay.

- **Dataset de Pokémon con texto**
  Se usa un dataset de Pokémon que incluye captions. Tiene 833 imágenes de 1280x1280. Las descripciones no son de gran calidad, pero es mejor que no tener nada. Se carga con `load_dataset` de la librería `datasets`, split de train.

- **Preprocesado de imágenes**
  Hay que adaptar las imágenes a las restricciones del modelo. Stable Diffusion v1.4 fue entrenado con resolución 512x512 (dato disponible en la descripción del modelo en Hugging Face). Transformaciones: 1) resize a 512x512, 2) conversión de imagen a tensor, 3) normalización. En redes neuronales el entrenamiento se estabiliza mejor con datos normalizados.

- **Clase wrapper del dataset**
  Se crea una clase que recibe un ejemplo del dataset, le aplica las transformaciones y devuelve los valores de los píxeles y los tokens del texto (`input_ids`). Sirve para que en el entrenamiento los datos lleguen ya transformados.

- **Qué componente fine tunear**
  Solo se fine tunea la UNet. El text encoder (CLIP) y el encoder/decoder de imágenes (VAE) son modelos pre-entrenados generalistas que ya funcionan bien para cualquier texto o imagen. Lo que se quiere cambiar es cómo el proceso de difusión predice y quita ruido: que en vez de generar patrones realistas, genere patrones estilo Pokémon. La UNet es el corazón del proceso de difusión, la que decide en qué dirección limpiar la imagen.

- **Carga de componentes por separado**
  Como solo se va a fine tunear la UNet, se borran el pipeline completo y la memoria de GPU, y se cargan los componentes de la tubería por separado.

- **Carga de componentes por separado**
  En lugar de usar `StableDiffusionPipeline` (que carga todo junto), se carga cada componente con su clase: `CLIPTokenizer` para el tokenizador, `CLIPTextModel` para el text encoder, `AutoencoderKL` para el VAE y `UNet2DConditionModel` para la UNet. Todas usan el metodo `from_pretrained` indicando el modelo base.

- **Congelar componentes que no se entrenan**
  Los componentes que no se van a entrenar se ponen en modo evaluacion (`.eval()`) y se establece `requires_grad = False` en todos sus parametros. Solo la UNet queda en modo entrenamiento.

- **UNet con entrada de texto**
  A diferencia del notebook anterior, esta UNet acepta input de texto: combina el input de texto con la imagen antes de entrar a la UNet. El dataset es el de Pokemon con imagenes y texto.

PontIA

- **Fine tuning clasico es lento**
  El fine tuning se podria hacer con el mismo bucle del notebook anterior, pero con este modelo mas grande seria muy lento.

> IDEA CLAVE
> **Solo se fine tunea la UNet, porque es el componente que decide en qué dirección quitar ruido; el text encoder y el VAE ya funcionan bien para cualquier entrada.**

> IDEA CLAVE
> **Para fine tunear solo la UNet se cargan los componentes del pipeline por separado y se congelan los que no se quieren entrenar.**

07 Funcion normalize y broadcasting

- **Parametros de normalize**
  La funcion `normalize` recibe dos listas: media y desviacion tipica, con un valor por canal. Para imagenes RGB (3 canales) se pasa `[0.5, 0.5, 0.5]` para media y `[0.5, 0.5, 0.5]` para desviacion tipica.
- **Broadcasting en PyTorch**
  Si se pasa un solo valor en vez de tres, PyTorch hace broadcasting (igual que NumPy) y lo aplica automaticamente a los tres canales. Es equivalente a escribir los tres valores.

08 Problemas del fine tuning clasico

- **Coste computacional**
  En modelos grandes (miles de millones de parametros) el fine tuning equivale casi a reentrenar el modelo desde cero: requiere muchisima GPU, tiempo y recursos. La idea del fine tuning es aprovechar un modelo entrenado por terceros y ajustarlo en un entorno menos ambicioso.
- **Riesgo de alterar el modelo original**
  Al ajustar todos los parametros se corre el riesgo de alterar demasiado el comportamiento original del modelo y no poder volver al estado inicial de forma sencilla.
- **Solucion: adapters**
  Para solventar estos problemas existen los adapters (adaptadores): componentes que se anaden a un modelo pre-entrenado. LoRA es un tipo concreto de adaptador que se explica a continuacion.
- **Un modelo por dominio**
  Si se quiere fine tunear para dos dominios distintos (por ejemplo, Pokémon y anime japonés), hay que hacer un fine tuning completo para cada uno. El resultado son dos modelos gigantescos con todos los parámetros ajustados. Para desplegarlos, se necesitan ambos modelos cargados y un selector que dirija cada petición al modelo correspondiente.

> IDEA CLAVE
> **El fine tuning clasico actualiza todos los parametros del modelo, lo que es ineficiente en modelos grandes y puede alterar demasiado su comportamiento original.**

PontIA

IDEA CLAVE

El fine tuning clásico obliga a mantener un modelo gigante completo por cada dominio, lo que es ineficiente en recursos y difícil de revertir.

09 Adaptadores (adapters)

- **Concepto de adaptador**
  En lugar de duplicar el modelo completo, se mantiene el modelo base común y se entrenan componentes pequeños (adaptadores) que, al acoplarse al modelo, producen un comportamiento equivalente al fine tuning. Para desplegar dos estilos basta un modelo grande y dos adaptadores pequeños.
- **Calidad frente a eficiencia**
  Los adaptadores producen algo menos de calidad que un fine tuning completo, pero la pérdida es mínima. La ganancia en eficiencia (entrenamiento más rápido, sin duplicar modelos) compensa con creces esa pequeña pérdida.

IDEA CLAVE

Los adaptadores permiten mantener un único modelo base y añadirle componentes pequeños y entrenables que replican el efecto del fine tuning con mucha menos carga.

10 LoRA (Low-Rank Adaptation)

- **Actualización de pesos en entrenamiento**
  En cada iteración de entrenamiento, los parámetros nuevos son los anteriores más una variación calculada por backpropagation: W_nuevo = W_anterior + ΔW. Esto aplica igual en el fine tuning de la UNet, que es una red neuronal con capas de atención.
- **Dependencia lineal entre columnas**
  En modelos grandes, muchas columnas de las matrices de pesos son linealmente dependientes de otras: se puede llegar a una columna multiplicando otra por un escalar o sumando vectores. Esas columnas redundantes no aportan información extra, porque el modelo ya sabe hacer operaciones lineales.
- **Rango de la matriz**
  El rango es el número máximo de columnas (o filas) linealmente independientes de una matriz. En modelos grandes, el rango es mucho menor que el número total de columnas. Por eso la técnica se llama Low-Rank Adaptation: adapta las matrices a un rango más bajo.
- **Descomposición en dos matrices**
  En vez de actualizar todos los parámetros de la matriz W, LoRA descompone la actualización ΔW en el producto de dos matrices más pequeñas, B y A. Por ejemplo, en lugar de actualizar 9 valores de una matriz 3x3, se optimizan 6 valores (una matriz 3x1 y otra 1x3). Al multiplicar B por A se obtiene una matriz del mismo tamaño que W, que se puede sumar a los pesos originales.
- **Hiperparámetro R (rango)**
  R determina la dimensión intermedia de las matrices B y A. Un R grande reduce menos la matriz: el entrenamiento tarda más pero se pierde menos calidad. Un R pequeño hace el fine tuning más eficiente y rápido, pero altera menos el comportamiento del modelo original.

PontIA

- **Configuración en código**
  Se usa la clase `LoraConfig` para añadir el adaptador LoRA a la UNet. Se indica el rango (por ejemplo, 6), `lora_alpha` (un factor multiplicador escalar delante de B x A), la inicialización de los pesos de A y B, y `target_modules` para indicar a qué capas se aplica. Se aplica sobre todo a las capas de atención: K, Q, V y la capa de output.

- **Añadir el adaptador LoRA a la UNet**
  Se usa `unet.add_adapter` pasándole la configuración LoRA y un nombre (p. ej. "pokemon"). El nombre permite tener varios adaptadores y elegir cuál activar.

- **Reducción de parámetros entrenables**
  Con rango r = 6 se entrenan ~1 M de parámetros de los 860 M del modelo (0,14 %). Con r = 28 sube a 0,64 %; con r = 280 al 6 % aproximadamente.

- **Elección del rango r**
  No hay una norma clara: es prueba y error. Valores de referencia: bajos de 4 a 8, altos de 4 a 128. Se recomienda empezar con valores bajos, ver resultados y subir si el tiempo de entrenamiento lo permite.

> IDEA CLAVE
> **LoRA descompone la actualización de pesos en dos matrices pequeñas, reduciendo drásticamente los parámetros a entrenar con una pérdida de calidad mínima.**

> IDEA CLAVE
> **LoRA permite fine-tunear solo una fracción mínima de los parámetros (p. ej. 0,14 % de 860 M) con resultados notables; el rango r se elige por prueba y error.**

11 Bucle de fine tuning en difusión latente con texto

- **Estructura del bucle**
  Dos bucles anidados: el externo recorre las épocas y el interno recorre cada batch de imágenes del dataset. Se pone la UNet en modo `train`.

- **Codificación de la imagen**
  Cada imagen se pasa por el VAE con `vae.encode` para obtener su representación en el espacio latente. El VAE no se entrena, solo se usa en inferencia hacia adelante.

- **Factor de escalado del espacio latente**
  Este modelo concreto multiplica el espacio latente por una constante de 0,18215 que estabiliza el entrenamiento. Es una particularidad documentada en el repositorio del modelo original.

- **Ruido y timestep**
  Se genera ruido aleatorio con `torch.randn` del tamaño del vector latente. Se elige un timestep t aleatorio con `torch.randint`. Se añade el ruido al vector latente con `noise_scheduler.add_noise` en el timestep elegido.

- **Codificación del texto**
  Se pasan los input IDs de los tokens del texto por el text encoder, que devuelve el text embedding (encoder hidden states).

PontIA

- **Predicción, loss y actualización**
  Se pasa a la UNet el vector latente con ruido, el timestep y el embedding de texto. La UNet predice cuánto ruido hay. Se calcula el coste entre el ruido predicho y el ruido real, se hace propagación hacia atrás y actualización de parámetros.
- **Sintaxis vs. conceptos**
  La sintaxis concreta (PyTorch, TensorFlow, etc.) se puede resolver con documentación o agentes de código. Lo importante es interiorizar el concepto y el flujo: saber qué se hace en cada paso y por qué.
- **Tiempo de entrenamiento**
  Con 5 épocas tarda unos 15 minutos por época, unas 2 horas y media en total. Asumible para un modelo de este tamaño.

> IDEA CLAVE
> **El bucle es igual que en difusión básica, pero la imagen se codifica con el VAE al espacio latente y el texto se codifica con el text encoder antes de pasarlo todo a la UNet.**

12 Carga del modelo fine-tuneado para inferencia

- **Reconstrucción del pipeline**
  En `from_pretrained` se pasan los componentes por separado: la UNet fine-tuneada, y el VAE, text encoder y scheduler originales (no han sido alterados).
- **Activación del adaptador**
  El adaptador LoRA está desactivado por defecto. Se activa con `pipe.set_adapters` indicando el nombre (p. ej. "pokemon"). Si hay varios adaptadores, se elige cuál activar con esa línea.
- **Resultados con datos limitados**
  Con solo 800 imágenes y un LoRA del 0,16 % de parámetros, el modelo ya genera imágenes con estilo Pokémon ante el mismo prompt y semilla. Algunos prompts funcionan mejor que otros; no es un modelo perfecto, pero para un entrenamiento casero los resultados son buenos.

> IDEA CLAVE
> **Al reconstruir el pipeline se sustituye la UNet original por la fine-tuneada y se activa el adaptador LoRA con `set_adapters`.**

13 Tarea del módulo

- **Objetivo de la entrega**
  Hacer un fine tuning completo con LoRA de la misma versión de Stable Diffusion vista en clase, pero con un dataset distinto: ilustraciones de libros antiguos (4000 imágenes en Hugging Face).
- **Formato de entrega**
  Entregar un script o notebook de Python con el código del fine tuning. Plazo: 15 días desde la fecha de clase. Se entrega a través del campus, preferiblemente como repositorio público de GitHub (también se admite ZIP).
- **Particularidades del nuevo dataset**
  El dataset tiene varias columnas. Solo interesan dos: `1600px` (contiene la imagen) e `infoalt` (contiene el caption/texto de la imagen).

PontIA

- **Columnas y formato de imagen**
  Del dataset solo se usan dos columnas: la imagen y su caption. Las imágenes no son cuadradas, pero el modelo trabaja con 512x512, así que hay que aplicar `crop` y `resize` para adaptarlas.

- **Parámetros que afectan al entrenamiento**
  Cuatro variables principales influyen en el tiempo y la calidad del fine-tuning: 1) número de muestras de entrenamiento, 2) tamaño de batch, 3) número de épocas, 4) hiperparámetro R (rango del LoRA). En el ejemplo de clase con el dataset de Pokémon se usaron 800 imágenes, LoRA rank 6, batch size 3 y 5 épocas.

- **Experimentar con los parámetros**
  El código base es copiar y pegar de lo visto en clase, cambiando el dataset. La parte importante es jugar con los parámetros: probar más épocas, batches más grandes o más muestras de entrenamiento hasta conseguir los mejores resultados.

- **Subir el modelo a Hugging Face**
  Una vez terminado el fine-tuning, se sube el modelo entrenado y el LoRA al perfil personal de Hugging Face. La cuenta es gratuita. Se puede subir manualmente desde la interfaz o mediante código, siguiendo la documentación enlazada en el enunciado.

- **Comparativa de resultados**
  Se generan 3-5 imágenes con los mismos prompts en el modelo original y en el modelo fine-tuneado, y se entrega la comparativa para ver el efecto del fine-tuning.

- **Entregables**
  1) Notebook con el fine-tuning del modelo. 2) Archivo con el link al repositorio personal de Hugging Face con el modelo subido. 3) Documento o imágenes con la comparativa (mismo prompt, modelo original vs. fine-tuneado). Preferiblemente como repositorio de GitHub o, si no, como zip.

- **GPU local vs. Google Colab**
  Si se dispone de GPU local, se recomienda hacerlo en local con un script, ya que permite entrenar con más muestras y obtener mejores resultados. Si no, Google Colab es válido, pero en ese caso debe ser un notebook.

> IDEA CLAVE
> **La clave de la entrega es experimentar con los parámetros de entrenamiento y comparar las imágenes del modelo original frente al fine-tuneado.**

14 Vision Transformers

- **Generación vs. clasificación de imágenes**
  El módulo se ha centrado en modelos de generación de imágenes con arquitectura de difusión (U-Net). Pero para clasificar imágenes los modelos de difusión no sirven. Hasta ahora se usaban CNNs (Convolutional Neural Networks), pero desde la aparición de los transformers estos también se aplican a imágenes.

- **Qué es un Vision Transformer**
  Es exactamente la misma arquitectura transformer que se usa en texto, pero aplicada a imágenes. Está más relacionado con el módulo anterior (transformers para texto) que con el de difusión.

PontIA

- **De tokens a patches**
  En texto, cada token (palabra) entra en paralelo al transformer. En imágenes, se divide la imagen en patches (regiones), se aplanan formando una secuencia, y cada región se codifica a un vector numérico que entra al transformer. Es el equivalente al tokenizador en texto.
- **Ventaja del mecanismo de atención**
  Las capas de atención permiten capturar relaciones relevantes entre píxeles de distintas regiones, lo que mejora la codificación de la imagen frente a las CNNs. Es una forma mejor de representar las imágenes, igual que los transformers mejoraron la representación del texto.
- **Casos de uso típicos**
  Con imágenes, los casos de uso de Vision Transformers son principalmente clasificación y agrupación: detección de defectos en piezas de fábrica, clasificación de imágenes médicas, clasificación de productos en e-commerce. La mayoría de problemas con imagen se pueden reducir a clasificación.
- **Keras y TransformerEncoder**
  En Keras se puede usar `TransformerEncoderLayer`, que pide dimensiones y número de cabezas de atención. Es genérico: sirve tanto para texto como para imagen. Keras incluye ejemplos de uso con ambos tipos de dato.

> IDEA CLAVE
> **Un Vision Transformer es la misma arquitectura transformer del texto aplicada a imágenes, dividiendo la imagen en patches que forman una secuencia.**

PontIA